

AIGOAT 1.0 - Task 2

Monocular Depth Estimation from Single RGB Images

Competition Guidelines

February 7, 2026

Abstract

This document provides comprehensive guidelines for Task 2 of the AIGOAT 1.0 competition. Participants are challenged to train a monocular depth estimation model that predicts dense depth maps from single RGB images. The task focuses on learning scale-invariant depth representations and deploying efficient inference-ready models in ONNX format. Models will be evaluated on a held-out test set using standard depth estimation metrics.

Contents

1	Introduction to Monocular Depth Estimation	3
1.1	What is Depth Estimation?	3
1.2	Why Scale-Invariant Depth?	3
2	Task Description	3
2.1	Objective	3
2.2	Input and Output Specifications	3
3	Dataset Structure	4
3.1	Directory Organization	4
3.2	Dataset Size	4
3.3	File Naming Convention	4
4	ONNX Model Requirements	4
4.1	Model Specifications	4
5	Evaluation Metrics	5
5.1	Normalization Strategy	5
5.2	Primary Metric: RMSE	5
5.3	Composite Score	5
5.3.1	Accuracy Score	5
5.3.2	Size Score	5
5.3.3	Speed Score	6
5.4	Complete Scoring Example	6
6	Evaluation Environment	7
6.1	Data Preprocessing	7
7	Leaderboard and Evaluation	7
7.1	Submission Format	7
7.2	Public vs. Private Leaderboard	8
8	Submission Checklist	8

9	Resources and Links	9
9.1	Dataset Access	9
9.2	Starter Notebook	9

1 Introduction to Monocular Depth Estimation

1.1 What is Depth Estimation?

Depth estimation is a core computer vision task that aims to recover the per-pixel distance between the camera and the observed scene. Unlike stereo or multi-view methods, monocular depth estimation predicts depth from a *single RGB image*, relying purely on learned visual cues such as perspective, shading, texture gradients, and object semantics.

Monocular depth estimation is widely used in:

- Robotics and autonomous navigation
- Augmented and virtual reality
- 3D scene understanding
- Computational photography

1.2 Why Scale-Invariant Depth?

Absolute metric depth is often ambiguous when only a single image is available. As a result, this task focuses on **scale-invariant depth estimation**, where predictions are evaluated after normalization. This allows models to focus on relative geometry and scene structure rather than absolute distance calibration.

2 Task Description

2.1 Objective

Your Goal: Given a single RGB image of size 448×448 , predict a dense depth map of the same spatial resolution.

Mathematically, you must learn a function f_θ such that:

$$\hat{D} = f_\theta(I) \quad (1)$$

where:

- $I \in \mathbb{R}^{3 \times 448 \times 448}$ is the input RGB image
- $\hat{D} \in \mathbb{R}^{448 \times 448}$ is the predicted depth map

Both predictions and ground truth depth maps are normalized prior to evaluation to ensure scale invariance.

2.2 Input and Output Specifications

Data	Shape	Description
Input Image	(3, 448, 448)	RGB image (float32)
Depth Map	(448, 448)	Normalized depth (float32)

Table 1: Data dimensions for Task 2.

Note

Note: No camera intrinsics or metadata are provided. The task is purely monocular.

3 Dataset Structure

3.1 Directory Organization

The dataset is organized as follows:

```
1 dataset/  
2 |-- train/  
3 |   |-- 000001_image.png  
4 |   |-- 000001_depth_map.png  
5 |   |-- 000002_image.png  
6 |   |-- 000002_depth_map.png  
7 |   |-- ...
```

Listing 1: Dataset directory structure

3.2 Dataset Size

- **Training samples:** 9,200 image–depth pairs
- **Test samples:** 1,536 RGB images

3.3 File Naming Convention

Each training sample consists of:

- `{id}_image.png` — input RGB image
- `{id}_depth_map.png` — corresponding normalized depth map

Test samples contain only input images.

Warning

The test set does **NOT** include ground truth depth maps. Your ONNX model will be executed on our evaluation server to generate predictions for the test set.

4 ONNX Model Requirements

4.1 Model Specifications

Participants must submit a **single ONNX model file** with the following specifications:

- **Input tensor shape:** (8, 3, 448, 448)
- **Input type:** float32
- **Output tensor shape:** (8, 448, 448)
- **Output type:** float32
- **Fixed batch size:** 8
- **No dynamic axes**
- **No preprocessing inside the model**

Warning

Any violation of the input/output specification will result in a submission error. Ensure your model strictly adheres to these requirements.

5 Evaluation Metrics

5.1 Normalization Strategy

Before evaluation, both predictions and ground truth depth maps are normalized to the $[0, 1]$ range on a per-sample basis to ensure scale invariance. The normalization is performed as follows:

$$\hat{D}_{\text{norm}} = \frac{\hat{D} - \min(\hat{D})}{\max(\hat{D}) - \min(\hat{D}) + \epsilon} \quad (2)$$

where $\epsilon = 10^{-8}$ is a small constant for numerical stability.

5.2 Primary Metric: RMSE

The primary evaluation metric is **Root Mean Square Error (RMSE)** calculated on normalized depth predictions:

```

1 import numpy as np
2
3 def calculate_rmse(predictions, ground_truth):
4     """
5     Calculate Root Mean Square Error
6
7     Args:
8         predictions: Normalized depth predictions (N, H, W)
9         ground_truth: Normalized ground truth depth maps (N, H, W)
10
11     Returns:
12         RMSE value (lower is better)
13     """
14     mse = np.mean((predictions - ground_truth) ** 2)
15     rmse = np.sqrt(mse)
16     return float(rmse)

```

Listing 2: RMSE calculation

5.3 Composite Score

The final leaderboard ranking is determined by a **composite score** that balances accuracy, model size, and inference speed:

$$\boxed{\text{Score} = \text{Accuracy} \times \text{Size Score} \times \text{Speed Score}} \quad (3)$$

5.3.1 Accuracy Score

The accuracy component rewards models with lower RMSE:

$$\text{Accuracy Score} = \frac{4.0}{4.0 + (10 \times \text{RMSE})^2} \quad (4)$$

5.3.2 Size Score

The size component penalizes large models:

$$\text{Size Score} = \frac{50 - \text{Model Size (MB)}}{20} \quad (5)$$

This formula provides:

5.3.3 Speed Score

The speed component rewards faster inference:

$$\text{Speed Score} = 0.16 + \log_{10} \left(7 - \frac{2}{5.33} \times t \right) \quad (6)$$

where t is the median inference time (in seconds) over 5 measured runs on the evaluation server.

Note

Inference Protocol: Your model will undergo a warmup run (not scored), followed by 5 measured inference passes. The **median** of these 5 times will be used for scoring to ensure fairness and reduce variance.

5.4 Complete Scoring Example

```

1 import numpy as np
2
3 # Example results
4 rmse = 0.15
5 model_size_mb = 45.2
6 inference_time = 2.5 # seconds (median of 5 runs)
7
8 # Calculate component scores
9 accuracy_score = 4.0 / (4.0 + (10 * rmse)**2)
10 # accuracy_score = 4.0 / (4.0 + 2.25) = 0.640
11
12 size_score = (50.0 - model_size_mb) / 20.0
13 # size_score = (50.0 - 45.2) / 20.0 = 0.240
14
15 speed_score = 0.16 + np.log10(7.0 - (2/5.33) * inference_time)
16 # speed_score = 0.16 + log10(7.0 - 0.938) = 0.942
17
18 # Final composite score
19 total_score = accuracy_score * size_score * speed_score
20 # total_score = 0.640 * 0.240 * 0.942 = 0.145
21
22 print(f"Final Score: {total_score:.6f}")

```

Listing 3: Score calculation example

Warning**Important Considerations:**

- Models producing NaN or Inf values will be disqualified
- Models exceeding 5GB of GPU memory during inference will fail
- Inference time includes only the forward pass (not data loading)
- The evaluation uses ONNX Runtime on GPU (CUDA)
- model size must be less than 100 mb else it won't be evaluated.
- rmse must be less than 0.16 else your submission won't be evaluated.
- speed must be less than 10s as returned per submission
- true speed could only be computed after making a submission but you can benchmark model improvements on kaggle on your own as a reference.

6 Evaluation Environment

6.1 Data Preprocessing

Test images are preprocessed as follows before being fed to your model:

```
1 import numpy as np
2 from PIL import Image
3
4 # Load image
5 img = Image.open(image_path).convert('RGB')
6 img_array = np.array(img, dtype=np.float32)
7
8 # Normalize to [0, 1] range
9 if img_array.max() > 1.0:
10     img_array = img_array / 255.0
11
12 # Convert HWC to CHW format (channels first)
13 img_array = np.transpose(img_array, (2, 0, 1))
14
15 # Shape: (3, 448, 448)
```

Listing 4: Server-side preprocessing

Note

Your model should **not** perform additional preprocessing. The input will already be in the range [0, 1] with shape (batch_size, 3, 448, 448).

7 Leaderboard and Evaluation

7.1 Submission Format

Your submission must be:

- A single ONNX model file

The ONNX model will be executed on the evaluation server to generate predictions for the test set. You should **NOT** submit pre-computed depth maps.

7.2 Public vs. Private Leaderboard

Leaderboard	Test Samples	Purpose
Public	20% (~512 samples)	Monitor progress during competition
Private	80% (~1024 samples)	Final ranking (revealed at competition end)

Table 2: Leaderboard split for evaluation.

Note

The public leaderboard is meant for you to track your progress, but the **final ranking** will be determined by performance on the private leaderboard. This prevents overfitting to the public test set.

8 Submission Checklist

Before submitting, verify that:

Submission Code Example

```

1 # Submission toolkit
2
3 !git clone --depth 1 https://github.com/chater-marzougui/MLS-GOAT.git
4 !mv MLS-GOAT/GOAT .
5 !rm -rf MLS-GOAT
6
7 from GOAT import submit, leaderboard
8
9 team_name = "your_team_name"
10 team_password = "your_team_password"
11
12 # Submit your ONNX model
13 submit.challenge2(
14     "path/to/your/depth_model.onnx",
15     team_name,
16     team_password
17 )
18
19 # View the leaderboard
20 leaderboard.getLB_chall2(team_name)

```

Warning

Only ONNX files are accepted. Predicted depth maps should **not** be submitted manually.

9 Resources and Links

9.1 Dataset Access

The full dataset required for this task can be accessed at the following link:

- **Training Data:** [Dataset Link \(Click Here\)](#)

9.2 Starter Notebook

- **Starter Notebook:** [Starter Notebook Link \(Click Here\)](#)

Note

It is **strongly recommended** to begin with the starter notebook to ensure correct data loading, ONNX export, and submission procedures consistent with the evaluation server. The baseline model is intentionally simple and can be significantly improved in terms of architecture, training strategy, data augmentation, and loss functions.

Good luck with the competition!